《生物医学智能信息处理》实验二：聚类分析

2026 年 10 月 14 日

一、实验目的与数据

掌握 K-means、层次聚类的构建、评估与调参方法，尝试重采样一致性聚类，分析 CNV 数据的几何聚集与已知疾病类别是否一致。按照指导书和简短要求，报告包含方法、流程、结果及完整代码；结构和 Python 技术路线参考 Exp1 实验记录。

数据文件为 `HCC_CNV_data.mat`。`ma` 原始维度为 2475×225，转置后为 225×2475（行对应样本）；`sample` 为 225×2，第一列是样本 ID，第二列是类别。标签直接读取该列，与特征逐行对齐，不沿用 Exp1 的 `co` 编码，也不按 ID 前缀推断类别。

类别	样本数	全部特征为 2	组内不同向量数
healthy	32	26	5
HBV	67	53	4
Cirrhosis	36	25	5
HCC	90	42	41

质量检查：无缺失或非有限值，无重复 ID，无零方差特征，特征取值为 1～5。但全体样本仅有 **46 种不同特征向量**，146/225（64.89%）的样本所有特征均为 2，且这 146 例分属四类。组内不同向量数之和不能作为全体不同向量数，因为向量跨组重复。这些样本的 ID 不同，不直接视为错误记录或删掉；重复向量会影响聚类形态与评估。本实验无法判断其源于生物学相似、离散化还是其他上游处理。

二、处理流程

1. 加载与转置矩阵，核对 ID、标签、维度与有限性，记录重复特征向量。
2. 按特征进行 Z 分数标准化；PCA 使用完整 SVD，按累计方差选维数。
3. 主流程保留 **17 个主成分**，累计解释方差 **90.08%**；80%/95% 分别需 12/22 维。前两维仅解释 30.35%，只用于作图，实际聚类使用 17 维。
4. 扫描 K-means 的 k=2～8，并比较表示方法与初始化次数；扫描层次聚类的距离/连接方式；另用 PAM 比较不同距离；构建重采样一致性矩阵。
5. 计算内部指标与疾病标签外部指标，保存配置表、每个样本的簇归属和图形。

所有标准化、PCA、方差筛选及聚类拟合均不使用疾病标签。方差 top500 在**未标准化矩阵**上计算后排序，随后标准化并 PCA。总体为探索性全样本聚类，未设置独立测试集；固定 k=4 的 ARI 调参比较是对当前数据的事后描述，不代表对新病人的预测性能。

三、方法与评价指标

K-means：使用 k-means++ 初始化、Lloyd 迭代，最小化平方欧氏簇内距离；主扫描 `random_state=42`，`n_init=50`。标准 K-means 不能直接替换成任意距离，因此距离比较另外采用 **K-medoids/PAM**，以真实样本为中心，8 次初始化并穷举中心交换。它与 K-means 的目标函数不同，报告中分别列出。

层次聚类：欧氏/曼哈顿/余弦距离 × single/complete/average 连接，另加欧氏 Ward，共 10 种合法组合，均扫描 k=2～8。Ward 只用于欧氏距离。

一致性聚类：每次无放回抽取 80% 样本，K-means（`n_init=10`）拟合。对每个样本对，以同簇次数除以共同出现次数得到共识矩阵 C，在 1−C 上用 average 层次聚类。B=50/100/200 为同一 200 次序列的前缀，k=2～8。PCA 表示固定，重采样评价的是该表示下聚类的稳定性。

指标：轮廓系数 s=(b−a)/max(a,b)，a 为簇内平均距离，b 为最近其他簇平均距离，越高越好，单点簇按定义记为 0；DB 越低越好；RI 为样本对关系的一致比例；ARI 校正随机一致性，1 表示完全一致，近 0 表示接近随机基线，允许为负。另保存 NMI。RI/ARI 不需要把簇编号映射为疾病名称。

表中默认轮廓系数与 DB 均在同一个 **Z 分数 PCA90% 欧氏空间**计算，便于比较分区；各方案自身空间/距离的轮廓系数另存 `silhouette_fit`。疾病标签只用于 RI、ARI、NMI。

四、实验结果：簇数选择

k	轮廓系数	DB	RI	ARI	簇大小
2	0.8088	1.8555	0.3417	-0.0273	205/20
3	0.8023	1.6232	0.3469	-0.0301	203/20/2
4	0.8019	1.1407	0.3469	-0.0302	203/20/1/1
5	0.8037	1.0669	0.3363	-0.0326	206/15/2/1/1
6	0.8003	1.0596	0.3457	-0.0339	203/16/2/2/1/1
7	0.7995	0.8229	0.3334	-0.0326	207/12/2/1/1/1/1
8	0.8049	0.7786	0.3359	-0.0343	206/12/2/1/1/1/1/1

在主流程 K-means 扫描中，轮廓系数最高为 **k=2，0.8088**，而 DB 最低为 k=8，0.7786，两项指标的选择并不一致。采用轮廓系数时选 k=2；保留 k=4 作为已知四类疾病的对照。四簇 ARI 为 −0.0302，不能恢复疾病分组。

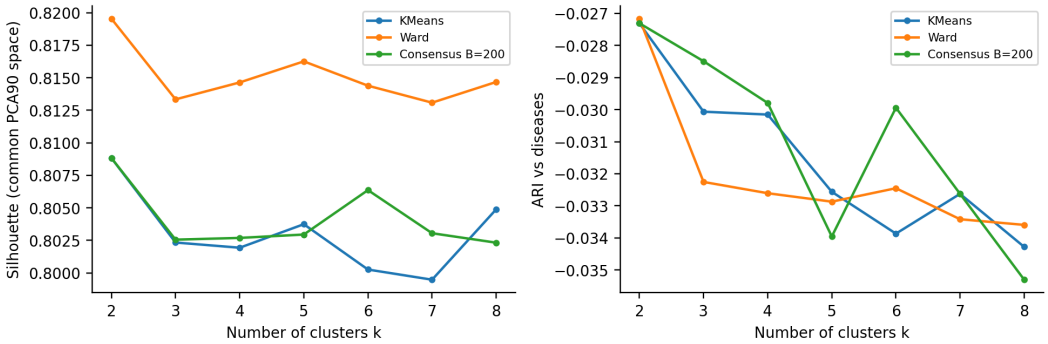


图 1: K-means、Ward 和一致性聚类的 k 扫描。左图为统一空间轮廓系数，右图为疾病 ARI，纵轴范围不同。

五、表示方法与距离参数调优（k=4）

表示方法	统一空间轮廓系数	ARI
PCA80%	0.8002	-0.0358
PCA90%	0.8019	-0.0302
PCA95%	0.8019	-0.0302
全特征 Z 分数	0.8023	-0.0317
原始值 PCA90%	0.7755	-0.0378
方差 top500+PCA90%	0.6253	-0.0217

PCA95% 与 PCA90% 在本次运行中给出相同分区，故统一空间指标相同；它们的自身空间轮廓系数分别为 0.7929 和 0.8019。方差筛选使 ARI 略有变化，但仍为负，未得到可靠的疾病分组。不能从 Exp1 的片段模式数据结果推断 CNV 数据也有效。

层次聚类的参数比较：

样本距离	连接方式	统一空间轮廓系数	ARI
euclidean	single	0.8388	-0.0055
euclidean	complete	0.8318	-0.013
euclidean	average	0.8388	-0.0055
euclidean	ward	0.8146	-0.0326
manhattan	single	0.8317	-0.0079
manhattan	complete	0.8204	-0.0127
manhattan	average	0.8317	-0.0079
cosine	single	0.5109	0.0059
cosine	complete	0.6554	-0.0162
cosine	average	0.6571	-0.0282

欧氏 single/average 的轮廓系数较高，但形成 222/1/1/1 的极端分区。最高疾病 ARI 是余弦 single 的 **0.0059**，簇大小为 150/73/1/1，仍接近随机基线。这里“最高”只指测试配置中的相对最高值。

PAM 的距离比较：

PAM 距离	原距离轮廓系数	统一空间轮廓系数	ARI
euclidean	0.5635	0.5635	-0.015
manhattan	0.5499	0.5366	-0.014
cosine	0.8212	0.6954	-0.0058

PAM 的余弦距离结果优于其欧氏/曼哈顿方案的 ARI，但仍为 −0.0058。余弦距离的 0.8212 与欧氏空间的 0.6954 度量不同，不能直接当成算法优劣差距。

六、随机初始化与一致性聚类

K-means 初始化稳定性：固定主流程表示与 k=4，对每个 `n_init` 使用种子 0～19，计算 20 次分区之间全部 190 对 ARI。这里的“运行间 ARI”比较不同运行的分区，区别于与疾病标签比较的 ARI。

初始化次数	运行间 ARI 均值	运行间 ARI 最小值	惯性标准差
1	0.4119	-0.0135	22004.2
10	0.8879	0.706	6787.22
50	0.937	0.803	1626.77

增加初始化次数，将运行间 ARI 均值从 0.4119 提高至 0.9370，惯性标准差也下降。因此重复初始化确实改善稳定性，但 `n_init` 是从多次初始化中选择最小惯性结果，**不等同于构建一致性矩阵**。

一致性聚类重采样次数比较：

重采样次数	PAC	轮廓系数	疾病 ARI	簇大小
50	0.1529	0.8019	-0.0302	203/20/1/1
100	0.1135	0.8019	-0.0302	203/20/1/1
200	0.112	0.8027	-0.0298	202/21/1/1

PAC 在本实验定义为非对角样本对中 0.1<C<0.9 的比例，越低表示模糊共聚类的样本对越少。B=200 时，任意样本对至少共同出现 100 次；B=50/100 的分区相同，B=200 有小幅变化，但疾病 ARI 仍为 −0.0298。本实验 k=2～8 的 B=200 PAC 从 0.1750 降至 0.0863，因此也不能仅凭 PAC 越小，就把更大的 k 认定为真实疾病簇数。

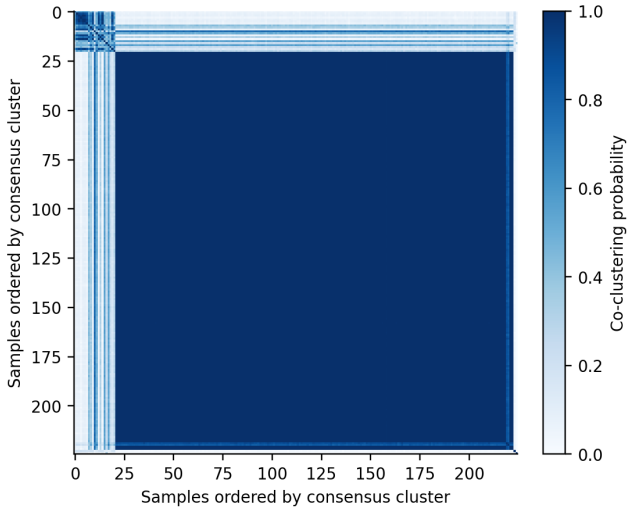


图 2: k=4、B=200 的一致性矩阵。样本按共识簇排序，同簇内再按疾病类别和特征向量排序；深色代表频繁共同聚类。

七、结果汇总与分析

固定 4 簇：各方法最高 ARI 方案	轮廓系数	RI	ARI	簇大小
层次（cosine/single）	0.5109	0.4831	0.0059	150/73/1/1
PAM（cosine）	0.6954	0.4983	-0.0058	151/42/26/6
K-means（方差 top500）	0.6253	0.4793	-0.0217	156/44/14/11
一致性（B=200）	0.8027	0.3502	-0.0298	202/21/1/1

上述表格按每种方法在 k=4 时的**疾病 ARI** 选出代表方案，是标签辅助的事后比较。它没有改变聚类拟合本身，也不能当作无偏的泛化性能估计。

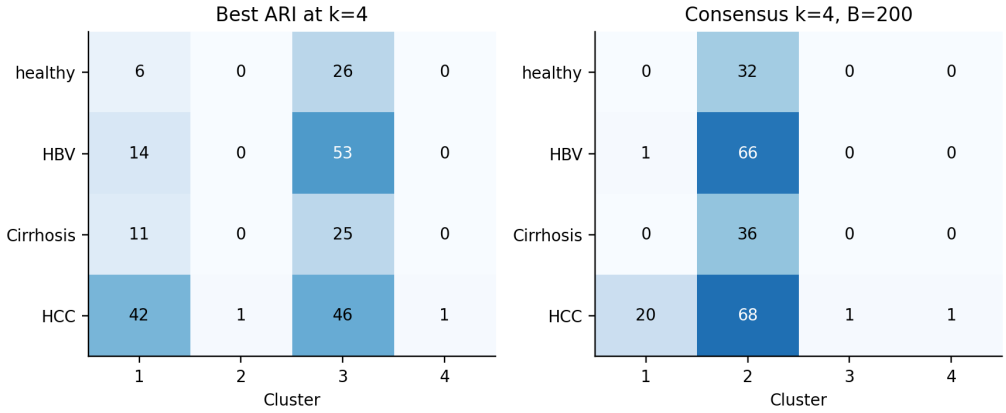


图 3: 左：k=4 时最高疾病 ARI 方案的疾病 × 簇列联表；右：一致性聚类结果。簇编号无疾病语义，不同图的编号不对应。

为何内部指标高而疾病一致性低？ 146 个相同向量涵盖健康 26、HBV 53、肝硬化 25、HCC 42 例。本次全部 136 个配置均将相同向量保持在同一簇，因此无法把这些样本分别放入正确的疾病组。高轮廓系数主要反映大簇与少数偏离样本的几何分离；全配置中最高统一空间轮廓系数 0.8646 对应欧氏 average 的 k=2，簇大小为 **224/1**，ARI 仅 −0.0027，更说明需同时查看簇大小与外部指标。

实验结论：在本数据及已测试参数范围内，K-means、层次聚类、PAM 和一致性聚类均未恢复四类疾病分组。距离、降维阈值和初始化次数会改变分区，其中多次初始化改善稳定性，但稳定的几何簇不等于有效的疾病分型。即使固定 k=4 时最高 ARI 为正，其数值 0.0059 也不足以支持“分型成功”。

局限：仅分析一个离散 CNV 矩阵；重复向量、类别不平衡及 PCA 以总体方差为目标会限制解释。未获得 CNV 特征的基因组位置、批次信息或上游处理说明，不能解释具体生物学机制，也不能据此否定 CNV 的全部诊断价值。后续可核查原始连续信号和批次，结合其他分子/临床信息，并在独立样本中验证；本次没有把这些建议写成已完成实验。

附录：完整实验代码

```
# -*- coding: utf-8 -*-
""" 实验二：CNV 聚类。运行 python Exp2/exp2_main.py；标签只用于评价。 """
from pathlib import Path
import hashlib
import json
import platform
import time

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import scipy
from scipy.cluster.hierarchy import dendrogram, linkage
from scipy.io import loadmat
import sklearn
from sklearn.cluster import AgglomerativeClustering, KMeans
from sklearn.decomposition import PCA
from sklearn.metrics import (adjusted_rand_score, davies_bouldin_score,
                             normalized_mutual_info_score, pairwise_distances,
                             rand_score, silhouette_score)
from sklearn.preprocessing import StandardScaler
from threadpoolctl import threadpool_limits

ROOT = Path(__file__).resolve().parent
OUT = ROOT / "results"
FIG = OUT / "figures"
TAB = OUT / "tables"
SEED = 42
GROUPS = ["healthy", "HBV", "Cirrhosis", "HCC"]
COLORS = ["#4477AA", "#228833", "#CCBB44", "#CC6677"]


def save_table(frame, name):
    frame.to_csv(TAB / name, index=False, encoding="utf-8-sig")


def save_figure(fig, name):
    fig.savefig(FIG / (name + ".png"), dpi=220, bbox_inches="tight")
    fig.savefig(FIG / (name + ".pdf"), bbox_inches="tight")
    plt.close(fig)


def pam(distance, candidates, k=4, repeats=8):
    """PAM SWAP：枚举每个 medoid 与非 medoid 的交换，取总距离最小者。
    候选来自不同原始特征向量，避免把重复样本作为多个初始中心。
    """
    rng = np.random.default_rng(SEED)
    best = None
    for _ in range(repeats):
        medoids = rng.choice(candidates, k, replace=False)
        cost = distance[:, medoids].min(axis=1).sum()
        for iteration in range(100):
            new_cost, new_medoids = cost, medoids.copy()
            for slot in range(k):
                other = np.delete(medoids, slot)
                nearest = distance[:, other].min(axis=1)
                costs = np.minimum(nearest[:, None], distance[:, candidates])
                costs = costs.sum(axis=0)
                costs[np.isin(candidates, medoids)] = np.inf
                j = int(np.argmin(costs))
                if costs[j] < new_cost - 1e-10:
                    new_cost = float(costs[j])
                    new_medoids = medoids.copy()
                    new_medoids[slot] = candidates[j]
            if new_cost >= cost - 1e-10:
                break
            medoids, cost = new_medoids, new_cost
        else:
            raise RuntimeError("PAM 达到迭代上限，请检查收敛")
        labels = distance[:, medoids].argmin(axis=1)
        if best is None or cost < best[0]:
            best = (float(cost), labels, medoids)
    return best


def consensus_runs(z, k, checkpoints=(50, 100, 200)):
    """80% 无放回重采样；同簇次数/共同出现次数，不把未抽到视为异簇。 """
    n = len(z)
    rng = np.random.default_rng(SEED + k)
    seen = np.zeros((n, n), dtype=np.int32)
    together = np.zeros_like(seen)
    outputs = {}
    for b in range(1, max(checkpoints) + 1):
        ix = np.sort(rng.choice(n, int(np.ceil(.8 * n)), replace=False))
```

```
        labels = KMeans(k, n_init=10, random_state=SEED + b).fit_predict(z[ix])
        block = np.ix_(ix, ix)
        seen[block] += 1
        together[block] += labels[:, None] == labels[None, :]
        if b in checkpoints:
            assert seen.min() > 0, " 存在从未共同采样的样本对"
            c = together / seen
            np.fill_diagonal(c, 1)
            d = np.clip(1 - c, 0, 1)
            labels = AgglomerativeClustering(
                n_clusters=k, metric="precomputed", linkage="average"
            ).fit_predict(d)
            upper = c[np.triu_indices(n, 1)]
            pac = float(np.mean((upper > .1) & (upper < .9)))
            outputs[b] = (labels, c.copy(), pac, int(seen.min()))
    return outputs


def main():
    start = time.perf_counter()
    threadpool_limits(1)  # 小样本单线程，避免过量线程开销并提高可重复性。
    for path in [FIG, TAB]:
        path.mkdir(parents=True, exist_ok=True)
    plt.rcParams.update({"font.family": "DejaVu Sans", "font.size": 9,
                         "axes.spines.top": False, "axes.spines.right": False,
                         "pdf.fonttype": 42})
    data_path = ROOT / " 实验 2-数据" / "HCC_CNV_data.mat"
    m = loadmat(data_path)
    x = np.asarray(m["ma"], dtype=float).T
    sample = m["sample"]
    ids = [str(v[0]).strip() for v in sample[:, 0]]
    groups = np.array([str(v[0]).strip() for v in sample[:, 1]])
    assert x.shape == (225, 2475) and sample.shape == (225, 2)
    assert np.isfinite(x).all() and len(set(ids)) == len(ids)
    assert set(groups) == set(GROUPS)
    y = np.array([GROUPS.index(g) for g in groups])
    unique, candidates, inverse, counts = np.unique(
        x, axis=0, return_index=True, return_inverse=True, return_counts=True)
    all_two = np.all(x == 2, axis=1)
    qc = pd.DataFrame({"sample_id": ids, "group": groups,
                       "profile_id": inverse, "profile_size": counts[inverse],
                       "all_CNV_2": all_two, "altered_features": (x != 2).sum(1)})
    save_table(qc, "sample_quality.csv")
    group_qc = qc.groupby("group", sort=False).agg(
        n=("group", "size"), all_CNV_2=("all_CNV_2", "sum"),
        unique_profiles=("profile_id", "nunique"))
    save_table(group_qc.reset_index(), "group_quality.csv")
    scaled = StandardScaler().fit_transform(x)
    pca = PCA(svd_solver="full").fit(scaled)
    scores = pca.transform(scaled)
    cumulative = np.cumsum(pca.explained_variance_ratio_)
    dims = {str(t): int(np.searchsorted(cumulative, t) + 1)
            for t in [.8, .9, .95]}
    z = scores[:, :dims["0.9"]]
    reps = {"zscore_PCA80": scores[:, :dims["0.8"]],
            "zscore_PCA90": z, "zscore_PCA95": scores[:, :dims["0.95"]],
            "zscore_all": scaled,
            "raw_PCA90": PCA(.9, svd_solver="full").fit_transform(x)}
    top = np.argsort(-x.var(axis=0), kind="stable")[:500]
    reps["variance500_PCA90"] = PCA(.9, svd_solver="full").fit_transform(
        StandardScaler().fit_transform(x[:, top]))
    save_table(pd.DataFrame({"PC": np.arange(1, len(cumulative) + 1),
                             "variance_ratio": pca.explained_variance_ratio_,
                             "cumulative": cumulative}), "pca_variance.csv")
    rows, partitions = [], {}
    reference_distance = pairwise_distances(z)

    def record(key, method, representation, k, labels, distmat, **extra):
        # 所有方案都在同一个 PCA90 欧氏空间复评，避免跨空间直接比较。
        assert len(labels) == len(x) and len(np.unique(labels)) == k
        partitions[key] = labels
        size = np.bincount(labels)
        row = dict(config=key, method=method, representation=representation,
                   k=k, silhouette_fit=silhouette_score(
                       distmat, labels, metric="precomputed"),
                   silhouette_reference=silhouette_score(
                       reference_distance, labels, metric="precomputed"),
                   DB_reference=davies_bouldin_score(z, labels),
                   RI=rand_score(y, labels), ARI=adjusted_rand_score(y, labels),
                   NMI=normalized_mutual_info_score(y, labels),
                   min_cluster=int(size.min()), max_cluster=int(size.max()),
                   cluster_sizes="/".join(map(str, sorted(size, reverse=True))))
        row.update(extra)
        rows.append(row)

    print("QC:", dict(group_qc["n"]), "unique profiles:", len(unique), flush=True)
    print("PCA dimensions:", dims, flush=True)
    for name, features in reps.items():
```

```
    distance = pairwise_distances(features)
    for k in range(2, 9):
        model = KMeans(k, n_init=50, random_state=SEED).fit(features)
        record(f"KM_{name}_k{k}", "KMeans", name, k,
               model.labels_, distance, inertia=float(model.inertia_))
print("KMeans sweep complete", flush=True)
for metric in ["euclidean", "manhattan", "cosine"]:
    distance = pairwise_distances(z, metric=metric)
    linkages = ["single", "complete", "average"]
    if metric == "euclidean":
        linkages.append("ward")  # Ward 仅适用于欧氏距离。
    for mode in linkages:
        for k in range(2, 9):
            labels = AgglomerativeClustering(
                n_clusters=k, metric=metric, linkage=mode).fit_predict(z)
            record(f"HC_{metric}_{mode}_k{k}", "Hierarchical",
                   "zscore_PCA90", k, labels, distance,
                   distance=metric, linkage=mode)
    objective, labels, medoids = pam(distance, candidates)
    record(f"PAM_{metric}_k4", "KMedoids", "zscore_PCA90", 4,
           labels, distance, distance=metric, objective=objective,
           medoid_ids="/".join(ids[i] for i in medoids))
print("Hierarchy and PAM complete", flush=True)
stability = []
for n_init in [1, 10, 50]:
    runs = [KMeans(4, n_init=n_init, random_state=s).fit(z)
            for s in range(20)]
    pair_ari = [adjusted_rand_score(a.labels_, b.labels_)
                for i, a in enumerate(runs) for b in runs[i + 1:]]
    stability.append(dict(n_init=n_init, seeds=20,
                          pair_ARI_mean=float(np.mean(pair_ari)),
                          pair_ARI_min=float(np.min(pair_ari)),
                          inertia_mean=float(np.mean([r.inertia_ for r in runs])),
                          inertia_std=float(np.std([r.inertia_ for r in runs]))))
save_table(pd.DataFrame(stability), "kmeans_stability.csv")
consensus = {}
for k in range(2, 9):
    for b, (labels, c, pac, min_seen) in consensus_runs(z, k).items():
        key = f"CC_B{b}_k{k}"
        record(key, "Consensus", "zscore_PCA90", k, labels,
               reference_distance, B=b, PAC=pac, min_pair_seen=min_seen)
        if b == 200:
            consensus[k] = c
    print(f"Consensus k={k} complete", flush=True)
table = pd.DataFrame(rows)
save_table(table, "all_metrics.csv")
chosen = table.loc[table.k == 4].sort_values(
    ["ARI", "silhouette_reference", "config"], ascending=[False, False, True])
best4 = chosen.iloc[0]
best_sil = table.sort_values(
    ["silhouette_reference", "config"], ascending=[False, True]).iloc[0]
family = chosen.groupby("method", sort=False).head(1)
save_table(family, "best_k4_by_ARI.csv")
# 一次组装宽表，避免 pandas 碎片化；簇编号仅用于显示，没有疾病语义。
labels_df = pd.concat([qc[["sample_id", "group"]],
                       pd.DataFrame({k: v + 1 for k, v in partitions.items()})],
                      axis=1)
save_table(labels_df, "cluster_assignments.csv")
np.savez_compressed(OUT / "model_arrays.npz", PCA90=z, PC2=scores[:, :2],
                    truth=y, **{f"consensus_k{k}": c
                                for k, c in consensus.items()})

fig, ax = plt.subplots(figsize=(6.5, 3.1))
ax.plot(np.arange(1, 41), cumulative[:40], color=COLORS[0], lw=2)
for t, dim in dims.items():
    ax.scatter(dim, cumulative[dim - 1], s=25)
    ax.annotate(f"{float(t):.0%}: {dim} PCs", (dim, cumulative[dim - 1]),
                xytext=(5, -20), textcoords="offset points", fontsize=8)
ax.set(xlabel="Number of principal components", ylabel="Cumulative variance",
       ylim=(0, 1.03), xlim=(1, 40))
save_figure(fig, "fig01_pca")
fig, axes = plt.subplots(1, 2, figsize=(9, 3.1))
for label, keys in [
    ("KMeans", [f"KM_zscore_PCA90_k{k}" for k in range(2, 9)]),
    ("Ward", [f"HC_euclidean_ward_k{k}" for k in range(2, 9)]),
    ("Consensus B=200", [f"CC_B200_k{k}" for k in range(2, 9)])]:
    sub = table.set_index("config").loc[keys]
    for ax, metric in zip(axes, ["silhouette_reference", "ARI"]):
        ax.plot(sub.k, sub[metric], marker="o", ms=3, label=label)
for ax, title in zip(axes, ["Silhouette (common PCA90 space)", "ARI vs diseases"]):
    ax.set(xlabel="Number of clusters k", ylabel=title, xticks=range(2, 9))
    ax.legend(fontsize=7)
fig.tight_layout()
save_figure(fig, "fig02_k_sweep")
fig, axes = plt.subplots(1, 3, figsize=(10.5, 3.3), sharex=True, sharey=True)
for ax, labels, title in zip(
    axes, [y, partitions[best4.config], partitions["CC_B200_k4"]],
    ["Known disease groups", "Best ARI at k=4", "Consensus k=4, B=200"]):
```

```
        for g in np.unique(labels):
            ix = labels == g
            text = GROUPS[g] if ax is axes[0] else f"Cluster {g + 1}"
            ax.scatter(scores[ix, 0], scores[ix, 1], s=18, alpha=.65,
                       color=COLORS[g], label=text, edgecolors="none")
        ax.set(title=title, xlabel="PC1")
        ax.legend(fontsize=7, loc="upper left")
    axes[0].set_ylabel("PC2")
    fig.tight_layout()
    save_figure(fig, "fig03_scatter")
    fig, axes = plt.subplots(1, 2, figsize=(8, 3.4))
    for ax, key, title in zip(axes, [best4.config, "CC_B200_k4"],
                              ["Best ARI at k=4", "Consensus k=4, B=200"]):
        ct = pd.crosstab(pd.Categorical(groups, categories=GROUPS),
                         partitions[key] + 1, dropna=False)
        save_table(ct.rename_axis("group").reset_index(), f"contingency_{key}.csv")
        ax.imshow(ct.values, cmap="Blues", vmin=0, vmax=90, aspect="auto")
        for i in range(4):
            for j in range(4):
                value = int(ct.iloc[i, j])
                ax.text(j, i, str(value), ha="center", va="center",
                        color="white" if value > 50 else "black")
        ax.set(xticks=range(4), xticklabels=ct.columns,
               yticks=range(4), yticklabels=GROUPS, xlabel="Cluster", title=title)
    fig.tight_layout()
    save_figure(fig, "fig04_contingency")
    fig, ax = plt.subplots(figsize=(8, 3.2))
    tree = linkage(z, method="ward", metric="euclidean")
    dendrogram(tree, truncate_mode="lastp", p=20, ax=ax,
               show_leaf_counts=True, leaf_rotation=90, leaf_font_size=8,
               color_threshold=0, above_threshold_color=COLORS[0])
    ax.set(xlabel="Terminal groups (sample counts in parentheses)",
           ylabel="Ward merge distance")
    save_figure(fig, "fig05_dendrogram")
    fig, ax = plt.subplots(figsize=(5, 4.2))
    c = consensus[4]
    order = np.lexsort((inverse, y, partitions["CC_B200_k4"]))
    heat = ax.imshow(c[np.ix_(order, order)], vmin=0, vmax=1,
                     cmap="Blues", interpolation="nearest")
    ax.set(xlabel="Samples ordered by consensus cluster",
           ylabel="Samples ordered by consensus cluster")
    fig.colorbar(heat, ax=ax, label="Co-clustering probability", fraction=.046)
    fig.tight_layout()
    save_figure(fig, "fig06_consensus")
    metadata = dict(seed=SEED, samples=len(x), features=x.shape[1],
                    group_counts=dict(zip(*np.unique(groups, return_counts=True))),
                    unique_profiles=len(unique), all_CNV_2=int(all_two.sum()),
                    constant_features=int((x.var(0) == 0).sum()),
                    PCA_dimensions=dims, PC2_variance=float(cumulative[1]),
                    PCA90_variance=float(cumulative[dims["0.9"] - 1]),
                    representations={k: v.shape[1] for k, v in reps.items()},
                    n_configs=len(table), best_k4_ARI=best4.to_dict(),
                    best_reference_silhouette=best_sil.to_dict(),
                    software=dict(python=platform.python_version(),
                                  numpy=np.__version__, scipy=scipy.__version__,
                                  sklearn=sklearn.__version__, pandas=pd.__version__,
                                  matplotlib=matplotlib.__version__),
                    data_sha256=hashlib.sha256(data_path.read_bytes()).hexdigest(),
                    code_sha256=hashlib.sha256(Path(__file__).read_bytes()).hexdigest(),
                    elapsed_seconds=time.perf_counter() - start)
    # 无需把 numpy 标量留在 JSON；未使用的指标写为 null。
    clean = json.loads(json.dumps(metadata, default=lambda v: v.item()))
    def no_nan(v):
        if isinstance(v, dict):
            return {k: no_nan(x) for k, x in v.items()}
        if isinstance(v, list):
            return [no_nan(x) for x in v]
        return None if isinstance(v, float) and not np.isfinite(v) else v
    (OUT / "metrics.json").write_text(
        json.dumps(no_nan(clean), ensure_ascii=False, indent=2), encoding="utf-8")
    print("Completed:", len(table), "configs; best k4 ARI:",
          best4.config, best4.ARI, flush=True)


if __name__ == "__main__":
    main()
```